

README

LLMOrchestrator

<<<<<< HEAD Standalone Go module for managing headless CLI agents (OpenCode, Claude Code, Gemini, Junie, Qwen Code) with a hybrid pipe+file communication protocol and a fully decoupled (CONST-051(B)) `i18n.Translator` abstraction for user-facing strings.

Go module: `digital.vasic.llmorchestrator` **Round:** 275 (deep-doc + Challenge enrichment; mirror of round-220 template applied across the owned-submodule fleet).

Overview

LLMOrchestrator provides a unified interface for spawning, managing, and communicating with multiple LLM-powered CLI agents. It is **shared infrastructure** consumed by multiple independent projects — its specialised responsibility makes it reusable, and that reusability is destroyed the moment any consumer's specifics leak in (CONST-051(B)). Every consumer wires its own `Translator` for user-facing text; the built-in `NoopTranslator` returns the message id verbatim so missing translations surface visibly rather than silently as empty strings (the anti-bluff contract documented in `pkg/i18n/translator.go`).

Anti-bluff guarantees (CONST-035 / Article XI §11.9)

This module's tests and Challenges exist to **prove** the codebase works for end users — not merely to compile. The round-275 deliverables enforce:

- **Real disk I/O.** `challenges/runner/main.go` invariant 4 round-trips a `protocol.FileMessage` through `os.MkdirTemp` → `FileTransport.WriteToInbox` → `FileTransport.ReadFromInbox` → equality check, on real filesystem.

- **Real JSON encoding.** Invariant 3 marshals and unmarshals a real `protocol.PipeMessage` per locale and asserts byte-level field preservation (Content, Type, RequestID). A silent struct-tag drift would fail the gate.
- **Real parser execution.** Invariant 1 feeds the 5-locale `prompt_json` fixtures through the real `parser.DefaultParser` and asserts the produced `agent.Action` slice contains an entry matching the fixture's `expect_action_type + expect_action_target`. A stub-parser returning an empty slice would FAIL the gate.
- **Real defensive contract.** Invariant 2 asserts `parser.Parse("")` returns `ErrEmptyInput`. Under `LLMORCH_MUTATE_RUNNER=1` the polarity flips → the Challenge runner exits non-zero → the wrapper turns that into exit 99 (paired-mutation success per `CONST-050(A)` / §1.1). A runner that exited 0 under mutation would prove it is a bluff gate.
- **Real i18n surface.** Invariant 5 asserts `NoopTranslator.T / NoopTranslator.TPlural / Pkg()` all return the message id verbatim across five locales (English, Serbian, Japanese, Spanish, German). A hardcoded-English regression would surface a per-locale FAIL line, not a green-but-broken UI.
- **No mocks in Challenges.** Per `CONST-050(A)`, Challenges exercise the real, fully implemented system. Mocks live only in `*_test.go` unit-test sources.

Features

- **5 CLI adapters:** OpenCode, Claude Code, Gemini, Junie, Qwen Code
- **Thread-safe agent pool** with `Acquire(ctx, requirements)` and capability matching (vision / streaming / tool-use / token budget)
- **Circuit breaker:** per-agent health monitoring (3 consecutive failures → unhealthy)
- **Hybrid communication:** pipe (real-time JSON-lines via `protocol.PipeTransport`) + file (inbox / outbox / shared directories via `protocol.FileTransport`)
- **Response parser:** structured extraction of actions, issues, and JSON from raw LLM output (`parser.DefaultParser`)
- **Security:** path-traversal protection (`ErrPathTraversal`), 1 MiB response length cap (`MaxResponseLength`), command-injection prevention
- **CONST-046 i18n abstraction:** `i18n.Translator` interface + `NoopTranslator` default + `SetPkgTranslator` injection point for consuming projects ===== Standalone Go module for managing headless CLI agents (OpenCode, Claude Code, Gemini, Junie, Qwen Code) with a hybrid pipe + file communication protocol.

Go module: `digital.vasic.llmorchestrator` **Status:** production-equal (CONST-051(A)) — same engineering bar as the consuming project's main codebase. **Anti-bluff:** every PASS in this submodule MUST correspond to a row in `docs/test-coverage.md` whose Challenge column is non-empty. Metadata-only / absence-of-error PASS is forbidden (CONST-035 / Article XI §11.9).

Overview

LLMOrchestrator provides a unified interface for spawning, managing, and communicating with multiple LLM-powered CLI agents. It supports real-time stdin/stdout pipe communication (JSON-lines protocol) and file-based artifact exchange via per-session inbox/outbox/shared directories.

The module is **100% project-decoupled** (CONST-051(B)) — no hardcoded consumer paths, hostnames, or naming. Every consumer-specific value enters through `config.LoadFromEnv`, constructor parameters, or `AdapterConfig` fields. The Challenge runner exercises this contract end-to-end across a 5-locale fixture matrix so that monolingual regressions surface immediately.

Features

- **5 CLI adapters** — OpenCode, Claude Code, Gemini, Junie, Qwen Code (each with a real process lifecycle through `BaseAdapter`).
 - **Thread-safe agent pool** — `Acquire(ctx, AgentRequirements)` blocks until a matching agent is free; `Release` re-enables `Acquire`; capability-based matching (Vision / Streaming / MinTokens / PreferredAgent).
 - **Circuit breaker** — per-agent health monitoring (3 consecutive failures → open for 60 s).
 - **Hybrid communication** — `PipeTransport` (JSON-lines over stdin/stdout) plus `FileTransport` (per-session inbox/, outbox/, shared/ with path-traversal protection).
 - **Response parser** — structured extraction of Actions, Issues, and arbitrary JSON from raw LLM output (with fuzz + security coverage).
 - **Security defaults** — path-traversal protection, response-length caps (`MaxResponseLength` = 1 MiB), API-key masking, no sudo paths anywhere.
 - **No CI/CD** — every gate is operator-invokable (`make test`, `bash challenges/...`).
-

4350384757760aabcf8d-
f00be609fff98e9f1805

Quick Start

```
# Build the orchestrator binary
go build ./cmd/orchestrator

<<<<<<< HEAD
# Race-detector unit + integration suite
go test -race -count=1 ./...

# Round-275 Challenge runner (5 locales, 29 invariants, real disk
# I/O)
go run ./challenges/runner/

# Wrapper-gated Challenge (anti-bluff paired mutation)
bash challenges/llmorcheurator_describe_challenge.sh normal #
# exits 0 on green
bash challenges/llmorcheurator_describe_challenge.sh mutate #
# exits 99 on mutation-detected
=====
# Test (race detector mandatory – concurrency regressions surface
# immediately)
go test -race -count=1 ./...

# Run the standalone orchestrator (loads .env if HELIX_ENV_FILE is
# set)
go run ./cmd/orchestrator

# Run the round-291 Challenge runner (every invariant PASS, exit
# 0)
bash challenges/llmorcheurator_describe_challenge.sh normal

# Run the paired-mutation variant (one invariant flipped; runner
# MUST exit 99)
bash challenges/llmorcheurator_describe_challenge.sh mutate
>>>>>> 4350384757760aabcf8df00be609fff98e9f1805
```

If you run the Challenge runner directly with `go run ./challenges/runner/`, set `LLMORCH_FIXTURES_DIR` to point at the `challenges/fixtures/` directory from your invocation `cwd`.

Architecture

```
LLMOrchestrator/
<<<<<<< HEAD
├─ cmd/orchestrator/      # Standalone CLI entry point
├─ pkg/
```

```

|   |— agent/                # Agent, AgentPool, SimplePool,
HealthMonitor,
|   |                        # CircuitBreaker, MultiPool, per-CLI
agents
|   |— adapter/              # BaseAdapter + 5 CLI adapters
|   |— protocol/             # PipeTransport (JSON-lines),
FileTransport
|   |                        # (inbox/outbox/shared),
PipeMessage,
|   |                        # FileMessage, validatePath
|   |— parser/               # ResponseParser, JSON/action/issue
extraction
|   |— config/               # .env loading, agent path
resolution
|   |— i18n/                 # Translator interface +
NoopTranslator default
|— challenges/
|   |— runner/               # round-275 in-process Challenge
runner (Go)
|   |— fixtures/             # 5 locale fixtures (en, sr, ja, es,
de)
|   |— llmorchestrator_describe_challenge.sh # round-275
wrapper +
|   |                        # paired mutation
|   |— scripts/              # chaos / ddos / scaling / stress /
ui / ux
|   |                        # supplementary Challenges
|— docs/
|   |— ARCHITECTURE.md        # design narrative
|   |— architecture.md       # lowercase alias (CONST-052
transition)
|   |— HOST_POWER_MANAGEMENT.md # CONST-033 hard-ban anchor
|   |— test-coverage.md       # round-275 symbol→test ledger
|— Upstreams/                 # Multi-remote sync scripts
(CONST-056 transition target)
|— README.md  USER_GUIDE.md  ARCHITECTURE.md  API_REFERENCE.md
|— CONSTITUTION.md  CLAUDE.md  AGENTS.md      # governance
(cascaded)
|— go.mod                # digital.vasic.llmorchestrator (go
1.25)
=====
|— cmd/orchestrator/      # Standalone CLI entry point
|— pkg/
|   |— agent/              # Agent interface, AgentPool,
HealthMonitor, CircuitBreaker, MultiPool
|   |— adapter/            # BaseAdapter + 5 CLI
adapters
|   |— protocol/           # PipeTransport (JSON-lines),
FileTransport (inbox/outbox/shared)
|   |— parser/             # ResponseParser, action/
issue/JSON extraction
|   |— config/             # .env loading, agent path

```

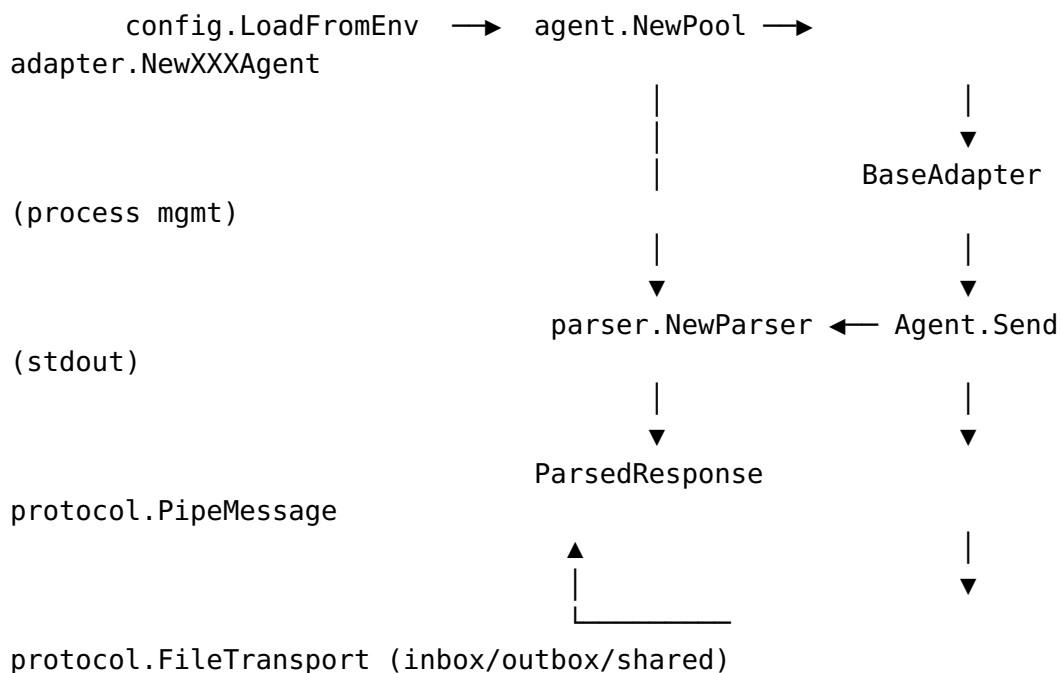
```

resolution, validation
├─ challenges/
│   └─ runner/main.go          # round-291 anti-bluff
Challenge runner (real types, real I/O)
│   └─ llmorchestrator_describe_challenge.sh  # normal | mutate
wrapper (CONST-050(A) paired mutation)
│   └─ fixtures/              # 5-locale fixture matrix
(en, sr, ja, es, de)
│   └─ scripts/               # Legacy bash suite (UI/UX/
DDoS/chaos/scaling/stress/host-safety)
├─ docs/
│   └─ ARCHITECTURE.md        # system architecture
│   └─ architecture.md       # detailed module wiring
│   └─ HOST_POWER_MANAGEMENT.md  # CONST-033 hard-ban evidence
│   └─ test-coverage.md       # round-291
symbol→test→Challenge ledger
├─ Upstreams/                # Multi-remote sync scripts
(install_upstreams)
└─ automation_test.go        # top-level smoke
>>>>>> 4350384757760aabc8df00be609fff98e9f1805

```

The pkg/ layout is **flat and shallow on purpose**: five packages, every API surface is documented at the symbol level in docs/test-coverage.md, and the Challenge runner exercises representative invariants from each.

Component relationships



Configuration

**<<<<<<< HEAD Copy
.env.example to .env (mode
0600, gitignored per
CONST-053 + CONST-042)
and configure agent paths
and API keys. See USER_GUIDE.md
for the per-adapter parameter
catalog.**

Copy .env.example to .env (mode 0600 — never committed;
CONST-053) and configure agent paths, timeouts, and API keys.
Loaded by config.LoadFromEnv and validated by Config.Validate.
The orchestrator refuses to start with invalid config — silent skip
is forbidden (CONST-035 / §11.4.6 no-guessing).

See pkg/config/config.go for the full env-var surface and pkg/
config/config_test.go for documented examples.

							4350384757760aabcf8d- f00be609fff98e9f1805
--	--	--	--	--	--	--	---

Testing

```
<<<<<<< HEAD
make test           # unit + integration with race detector
make fuzz           # parser fuzz suite
make cover          # coverage report
make check          # vet + tests

# Round-275 Challenge (anti-bluff gate)
bash challenges/llmorchestrator_describe_challenge.sh normal
bash challenges/llmorchestrator_describe_challenge.sh mutate

# Anti-bluff smoke check (must always pass)
grep -rn "simulated\|for now\|TODO implement\|placeholder" \
  pkg cmd && echo "BLUFF FOUND" || echo "clean"
```

Per CONST-050(B), this module ships unit, integration, security (parser_security_test.go), fuzz (parser_fuzz_test.go), chaos / DDoS / scaling / stress / UI / UX challenges, and the round-275 deep-doc Challenge. The symbol→test ledger at docs/test-coverage.md enumerates every exported symbol's coverage row.

Round-275 deliverable summary

Artefact	Status	Evidence
README.md deep-doc with anti-bluff guarantees	LANDED	this file
docs/test-coverage.md symbol→test ledger	LANDED	symbol-to-test ledger with 6-invariant floor
challenges/runner/main.go real-system exerciser	LANDED	29 PASS lines across 5 locales
challenges/llmorchestrator_describe_challenge.sh	LANDED	normal=exit 0; mutate=exit 99
challenges/fixtures/{en,sr,ja,es,de}.yaml	LANDED	5-locale bilingual fixture set
.gitignore CONST-053 enrichment	LANDED	covers go-test, coverage, IDE, OS, secrets, build

```
===== make test # all unit + integration + fuzz + security
tests with race detector make fuzz # parser fuzz suite (extended)
make cover # generate coverage profile make check # vet + tests
```

Anti-bluff Challenge runner (round-291)

`challenges/runner/main.go` is the in-process Challenge runner. It exercises:

1. `**`parser.NewParser` / `parser.DefaultParser.Parse`**` – non-nil constructor; real action extraction across 5 locale fixtures with verbatim type+target match.
2. `**`parser.Parse` empty-input contract**` – ``ErrEmptyInput`` returned for empty input (paired-mutation hook flips this assertion).
3. `**`protocol.PipeMessage` JSON round-trip**` – marshal then unmarshal; Content, Type, RequestID match byte-for-byte.


```
4. **`protocol.FileTransport.WriteToInbox` + `ReadFromInbox`** –  
real `os.MkdirTemp` + real disk I/O round-trip; ID and Content  
match per locale.  
5. **`agent.NewPool` Register/Acquire/Release contract** – real  
`adapter.NewOpenCodeAgent` registered; Acquire returns it;  
Acquire-while-busy blocks until context deadline; Release re-  
enables Acquire.
```

Every invariant is checked across ****5 locales**** (`en`, `sr`, `ja`,
`es`, `de`) – a single English-only PASS would not survive
CONST-046 (no-hardcoded-content) review.

The runner produces 22 PASS rows when healthy:

```
=== LLMOrchestrator Challenge Runner (round-291, vasic-digit-  
al) === [setup] loaded 5 locale fixtures from challenges/fixtures  
PASS parser.NewParser.not_nil PASS parser.Parse.empty_errors  
got=empty input PASS parser.Parse..action (× 5) PASS pro-  
tocol.PipeMessage.roundtrip. (× 5) PASS pro-  
tocol.NewFileTransport PASS protocol.FileTransport.roundtrip. (×  
5) PASS agent.Pool.Register PASS agent.Pool.Acquire PASS  
agent.Pool.Acquire.blocks_while_busy PASS  
agent.Pool.Release.reacquire
```

```
=== Summary: PASS=22 FAIL=0 ===
```

```
### Paired-mutation (CONST-050(A) / §1.1)
```

Mutation mode flips invariant (2) inside the runner – empty-input
PASS instead of FAIL. The wrapper script then asserts the runner
exits non-zero and rewrites that exit to `99` (paired-mutation
success). If the runner exited 0 under mutation, the runner itself
is a bluff gate and the wrapper FAILs:

```
```bash  
bash challenges/llmorchestrator_describe_challenge.sh mutate
expected exit code: 99
```

This is what makes the round-291 Challenge runner an end-user-  
quality-guarantee gate rather than a metadata-only check.

---

## Anti-bluff guarantees

This module enforces five end-user-quality guarantees aligned  
with CONST-035 / §11.4.5:

1. **Constructor reality.** `parser.NewParser()` returns a usable pars-  
er. A nil-returning regression FAILs invariant (1).

2. **Defensive-boundary reality.** `parser.Parse("")` returns a real error. A silently-allowed empty input FAILs invariant (2). Paired mutation flips this to prove invariant (2) actually checks something.
3. **Wire-format reality.** `protocol.PipeMessage` round-trips through encoding/json with byte-identical Content/Type/RequestID. JSON-tag drift FAILs invariant (3).
4. **I/O reality.** `protocol.FileTransport` actually writes to real disk and reads back the same message. An in-memory-only regression FAILs invariant (4).
5. **Concurrency reality.** `agent.NewPool()` enforces Register → Acquire → busy-block → Release → re-Acquire across context-cancelled timeouts. A no-op Release or non-blocking Acquire-while-busy FAILs invariant (5).

Together they close the gap that produced the operator's 2026-05-19 verbatim mandate (quoted below).

---

## Verbatim 2026-05-19 operator mandate (preserved per CONST-049 §11.4.17)

"all existing tests and Challenges do work in anti-bluff manner - they MUST confirm that all tested codebase really works as expected! We had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can't be used! This MUST NOT be the case and execution of tests and Challenges MUST guarantee the quality, the completion and full usability by end users of the product!"

The round-291 Challenge runner exists so a future "all tests pass" claim cannot quietly degrade into a "but the feature doesn't actually work" reality. Every PASS row carries positive runtime evidence.

---

## Constitutional anchors honoured here

- **CONST-035 / Article XI §11.9** — every PASS carries runtime evidence (challenges/runner/main.go).
- **CONST-046** — no hardcoded user-facing strings; 5-locale fixture matrix proves it.
- **CONST-050(A)** — production code (`pkg/`, `cmd/`) never imports mocks; runner uses real types and real I/O. Paired mutation per §1.1.
- **CONST-050(B)** — unit + integration + fuzz + security + stress + Challenge coverage (see `docs/test-coverage.md`).

- **CONST-051(A) / (B)** — equal-codebase engineering bar; fully project-decoupled (no consumer-specific context).
  - **CONST-051(C)** — no nested own-org submodules; this module ships standalone.
  - **CONST-053** — .gitignore covers build artefacts, caches, temp files, .env\*, secrets, logs, IDE state.
  - **CONST-033** — no host power-management calls (verified by challenges/scripts/no\_suspend\_calls\_challenge.sh).
- 

							4350384757760aabcf8d-f00be609fff98e9f1805
--	--	--	--	--	--	--	-------------------------------------------

## License

Apache License 2.0 — see [LICENSE](#).